

Callback, Fetch e Programmazione Asincrona

Una callback è una funzione passata come argomento a un'altra funzione, con l'intenzione che venga eseguita in un momento successivo, tipicamente al completamento di un'operazione o al verificarsi di un evento.

La `fetch` è un'operazione asincrona e non bloccante: quando viene invocata, il runtime JavaScript registra la richiesta e prosegue immediatamente con le istruzioni successive, senza attendere la risposta del server. Lo dimostra il primo esempio di codice negli appunti, in cui il `console.log("Caricamento in corso")` alla riga 7 viene eseguito prima ancora che la catena `.then()` abbia ricevuto i dati, proprio perché la `fetch` non blocca l'esecuzione. Questo comportamento si contrappone alla **programmazione sincrona**, in cui ogni istruzione attende il completamento di quella precedente. Il motivo per cui il caricamento dei dati dal backend è sempre asincrono risiede proprio nella necessità di non bloccare l'interfaccia utente: se l'operazione fosse sincrona, la pagina si congelerebbe e l'utente non potrebbe interagire con nulla durante l'attesa.

`fetch` restituisce una **Promise**, che è un oggetto JavaScript rappresentante il risultato futuro di un'operazione asincrona. (Una **Promise può andare a buon fine o no** può trovarsi in uno di tre stati: *pending*, quando l'operazione è ancora in corso; *fulfilled*, quando si è conclusa con successo; *rejected*, quando si è verificato un errore.) La gestione avviene tramite i metodi `.then()`, che riceve il valore restituito in caso di successo, e `.catch()`, che intercetta l'eventuale errore. Con la sintassi moderna `async/await`, lo stesso risultato si ottiene tramite un blocco `try/catch`, con una leggibilità superiore.

È importante notare che `fetch` rigetta la **Promise** esclusivamente in caso di errore di rete o di richiesta non completata: se il server risponde con un codice di errore HTTP come 404 o 500, la **Promise** viene comunque risolta perché tecnicamente la risposta è arrivata. Questo aspetto è illustrato nel terzo e quarto esempio di codice, dove la `fetch` verso un endpoint inesistente (`/api/pople?q=Mar`) riceve una risposta 404 ma la prima **Promise** risulta comunque mantenuta. Per forzare la propagazione dell'errore in questi casi, è necessario verificare esplicitamente `response.ok` all'interno del primo `.then()` e lanciare manualmente un'eccezione tramite `throw` se il valore è `false`: solo così il controllo viene trasferito al `.catch()` successivo. Il commento nel codice chiarisce ulteriormente il concetto: `fetch` produce una **Promise** che potrebbe contenere, se mantenuta, una **Response**, e la **Response** potrebbe a sua volta contenere un JSON.

La **Response** è un oggetto HTTP generico che rappresenta l'intera risposta del server: contiene metadati come lo status code, gli header e un body sotto forma di stream di dati grezzi non ancora letti. Il metodo `.json()` è solo uno dei modi per leggere quel body: lo interpreta come testo JSON e lo converte in un oggetto JavaScript, restituendo a sua volta una **Promise** perché la lettura dello stream è un'operazione asincrona. Il body può essere letto una sola volta. Altri metodi di lettura sono `.text()` per testo semplice o HTML, `.blob()` per dati binari e `.arrayBuffer()` per byte grezzi.

Ogni `.then()` restituisce una nuova **Promise**, rendendo possibile concatenare operazioni asincrone in sequenza: il valore restituito all'interno di un `.then()` viene passato come argomento al `.then()` successivo. L'ordine di esecuzione delle callback asincrone non è garantito: il momento in cui una callback passata a `.then()` o `.catch()` viene eseguita dipende da fattori esterni come la latenza di rete, il carico del server e la velocità di parsing, tutti elementi che il codice client non può controllare né prevedere.

Servizi, HttpClient e Observable in Angular

Un servizio è un oggetto condiviso all'interno dell'applicazione. Esiste una sottocategoria di servizi il cui compito specifico è comunicare con le API esterne, fungendo da intermediari con il database. In Angular, `provideHttpClient()` rende disponibile il servizio `HttpClient` in tutta l'applicazione, che è il meccanismo Angular per eseguire richieste HTTP in sostituzione del `fetch` nativo del browser.

Angular utilizza gli `Observable` al posto delle `Promise` per gestire operazioni asincrone come le chiamate HTTP. Un `Observable` proviene dalla libreria `RxJS` e rappresenta un flusso di dati asincrono a cui ci si può sottoscrivere per ricevere valori nel tempo.

Un `Observable` è per natura lazy: non esegue nulla finché non esiste almeno un subscriber. A differenza di una `Promise`, che rappresenta un singolo risultato futuro e si risolve una sola volta, un `Observable` può emettere zero, uno o molti valori in momenti diversi, concludendosi solo quando il flusso termina. Gli `Observable` vengono prodotti nei servizi e consumati nei componenti: è il componente che si occupa di gestire gli errori del proprio `Observable` tramite le callback passate a `.subscribe()`. Questo metodo riceve un oggetto con due callback: `next`, equivalente del `.then()` delle `Promise`, chiamata quando l'`Observable` emette un valore con successo; ed `error`, equivalente del `.catch()`, chiamata se si verifica un errore.